

Reviewer Pack — Free Cumulant Spread Bounds Disjointness Communication Compl...

Entry b13a492c0ac9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-13 02:43:47 UTC

Free Cumulant Spread Bounds Disjointness Communication Complexity

Entry ID: b13a492c0ac9 Verdict: INCONCLUSIVE Recorded: 2026-05-13 02:43:47 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

For a random 0-1 matrix $M \in \{0,1\}^{n \times n}$ representing disjointness, the free cumulant spread $\sigma(M) = \max_k |\kappa_k(M)| - \min_k |\kappa_k(M)|$ satisfies $\sigma(M) \geq \Omega(n)$, where $\kappa_k(M)$ is the k -th free cumulant of M 's entries treated as a noncommutative random variable.

Rationale (proposer's reasoning):

Free cumulants capture non-crossing partition dependencies in high-dimensional data, which may mirror the combinatorial structure of disjointness protocols. The spread reflects how noncommutative correlations scale with input size, potentially exposing lower bounds via free entropy methods.

Taxonomy category: COMM_DISJ (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 340258440f6d64ce

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=5.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_matrix(n):
        return [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]

    def moment_cumulant_inversion(moments):
        n = len(moments)
        cumulants = [moments[0]]
        for k in range(1, n):
            cumulant = moments[k]
            for i in range(k):
                cumulant -= sum(cumulants[i] * cumulants[j] for j in range(i)) / (i + 1)
            cumulants.append(cumulant)
        return cumulants

    def free_cumulant_spread(M):
        n = len(M)
        moments = [sum(sum(M[i][j] * M[k][l] for j in range(n) for l in range(n)) for i in range(n))
        cumulants = moment_cumulant_inversion(moments)
        return max(abs(c) for c in cumulants) - min(abs(c) for c in cumulants)

    n_values = [10, 20, 30, 40]
    results = []
    for n in n_values:
        M = generate_matrix(n)
        sigma_M = free_cumulant_spread(M)
        results.append(sigma_M >= 0.1 * n)

    metric_value = sum(results) / len(results)
    conjecture_holds = all(results)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "free_cumulant_spread",
        "metric_value": metric_value,
```

```

        "instances_tested": len(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value:.6f} std=0.000000 support_fraction={support_fraction:.6f}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={seeds[0]}")
    else:
        print("RESULT: INCONCLUSIVE insufficient seeds or support_fraction < 80%")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

"conjecture_holds": True, "counterexample": ""
TRIAL: {"seed": 421, "metric_name": "free_cumulant_spread", "metric_value": 1.000000, "instances_tested": 1000}
TRIAL: {"seed": 463, "metric_name": "free_cumulant_spread", "metric_value": 1.000000, "instances_tested": 1000}
TRIAL: {"seed": 503, "metric_name": "free_cumulant_spread", "metric_value": 1.000000, "instances_tested": 1000}
TRIAL: {"seed": 547, "metric_name": "free_cumulant_spread", "metric_value": 1.000000, "instances_tested": 1000}
TRIAL: {"seed": 593, "metric_name": "free_cumulant_spread", "metric_value": 1.000000, "instances_tested": 1000}
TRIAL: {"seed": 631, "metric_name": "free_cumulant_spread", "metric_value": 1.000000, "instances_tested": 1000}
TRIAL: {"seed": 677, "metric_name": "free_cumulant_spread", "metric_value": 1.000000, "instances_tested": 1000}
TRIAL: {"seed": 727, "metric_name": "free_cumulant_spread", "metric_value": 1.000000, "instances_tested": 1000}
TRIAL: {"seed": 773, "metric_name": "free_cumulant_spread", "metric_value": 1.000000, "instances_tested": 1000}
TRIAL: {"seed": 821, "metric_name": "free_cumulant_spread", "metric_value": 1.000000, "instances_tested": 1000}
TRIAL: {"seed": 877, "metric_name": "free_cumulant_spread", "metric_value": 1.000000, "instances_tested": 1000}
TRIAL: {"seed": 929, "metric_name": "free_cumulant_spread", "metric_value": 1.000000, "instances_tested": 1000}
RESULT: SUPPORTED mean=1.000000 std=0.000000 support_fraction=1.00

```

8. Critic adversarial review

Critic verdict: CHALLENGE

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

parse_fail | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111774
2	propose	ollama_remote	qwen3:8b	0	0	40619
3	preregistration	ollama_remote	qwen3:8b	0	0	24406
4	novelty	ollama_remote	qwen3:8b	0	0	20700
5	novelty	ollama_remote	qwen3:8b	0	0	13867
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12341
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8788
8	critic	ollama_remote	qwen3:8b	0	0	32212
9	judge	ollama_remote	qwen3:8b	0	0	21056

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 285764 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b13a492c0ac9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b13a492c0ac9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b13a492c0ac9.tar.gz` (if generated)